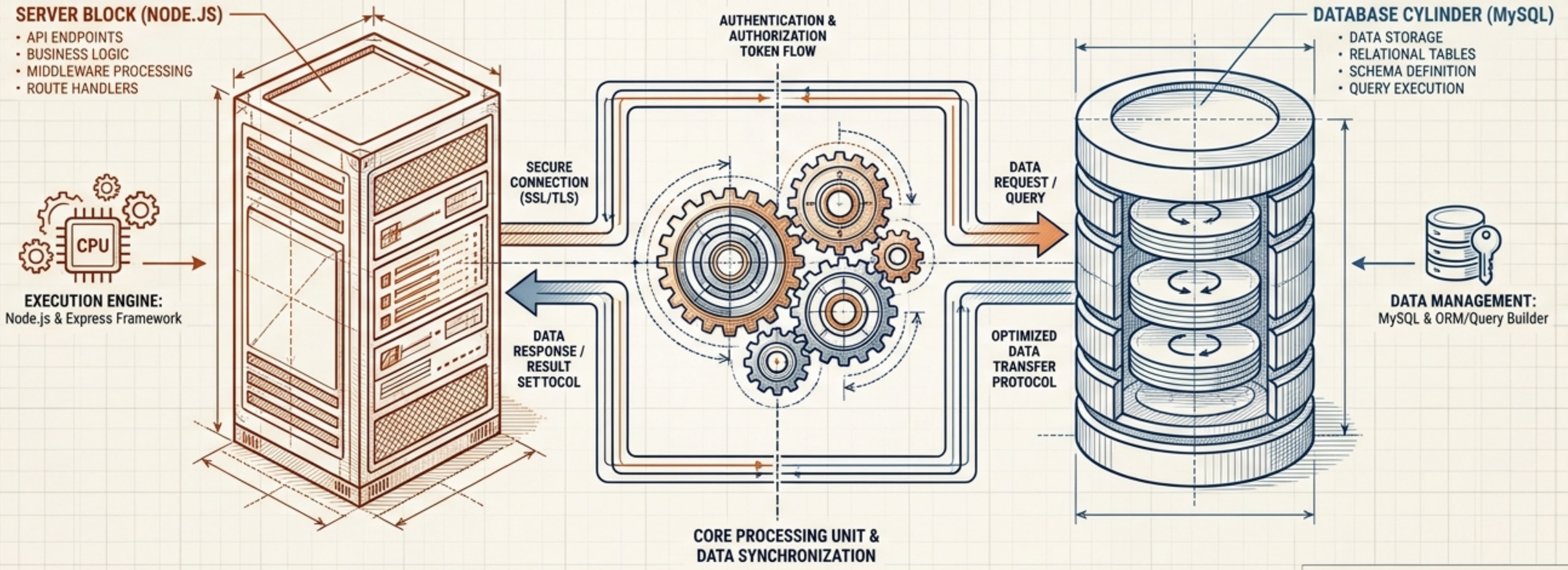
Construindo uma API REST Robusta

Anatomia do Back-end: De Bancos de Dados a Rotas Seguras com Node.js e MySQL (Estudo de Caso: Projeto Nutri DB)



Gemini Notebook

A Filosofia Primordial: Nunca Confie no Usuário

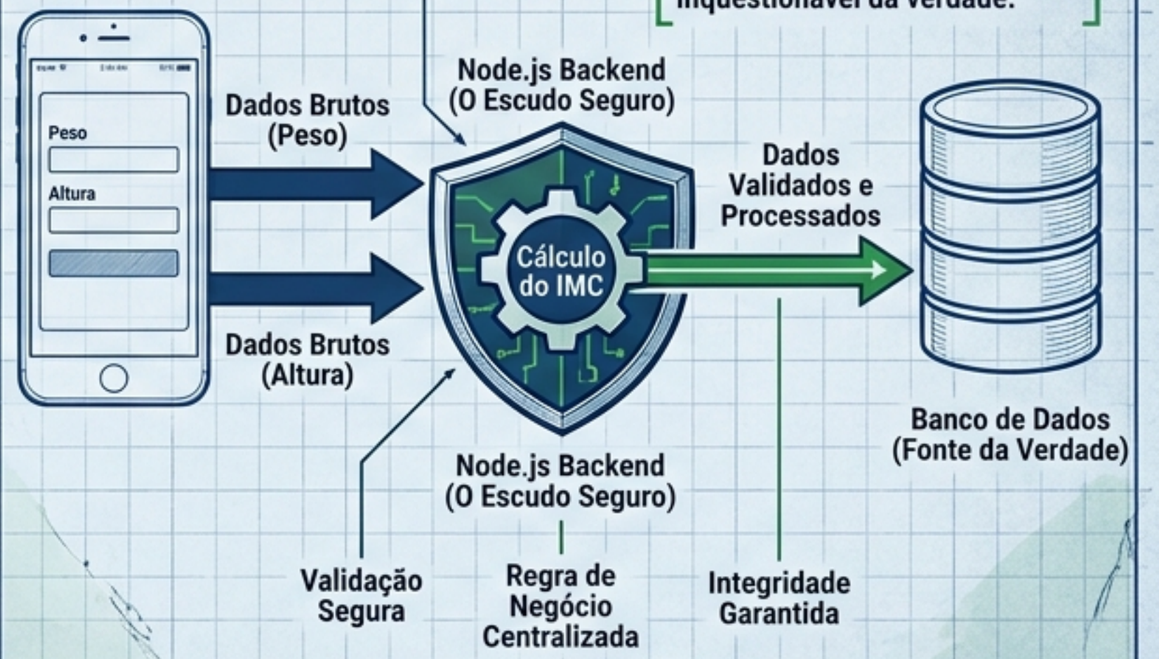
Por que a regra de negócio deve pertencer ao servidor.

O Caminho Inseguro

O Erro Comum: Deixar o cálculo matemático no front-end expõe o banco de dados a manipulações e dados corrompidos.

A Prática Robusta

O Escudo:
O front-end é apenas uma interface de envio. O back-end é o processador seguro e a fonte inquestionável da verdade.



Gemini Notebook

A Fundação de Dados (Nutri DB)

Estrutura relacional da tabela pacientes

pacientes	
id	[Primary Key]
nome	[VARCHAR]
altura	[DECIMAL(4,2)]
peso	[DECIMAL(4,2)]
imc	[DECIMAL(4,2)]
status	[VARCHAR]
criado_em	[TIMESTAMP]

Auto-incremento (Chave Primária).

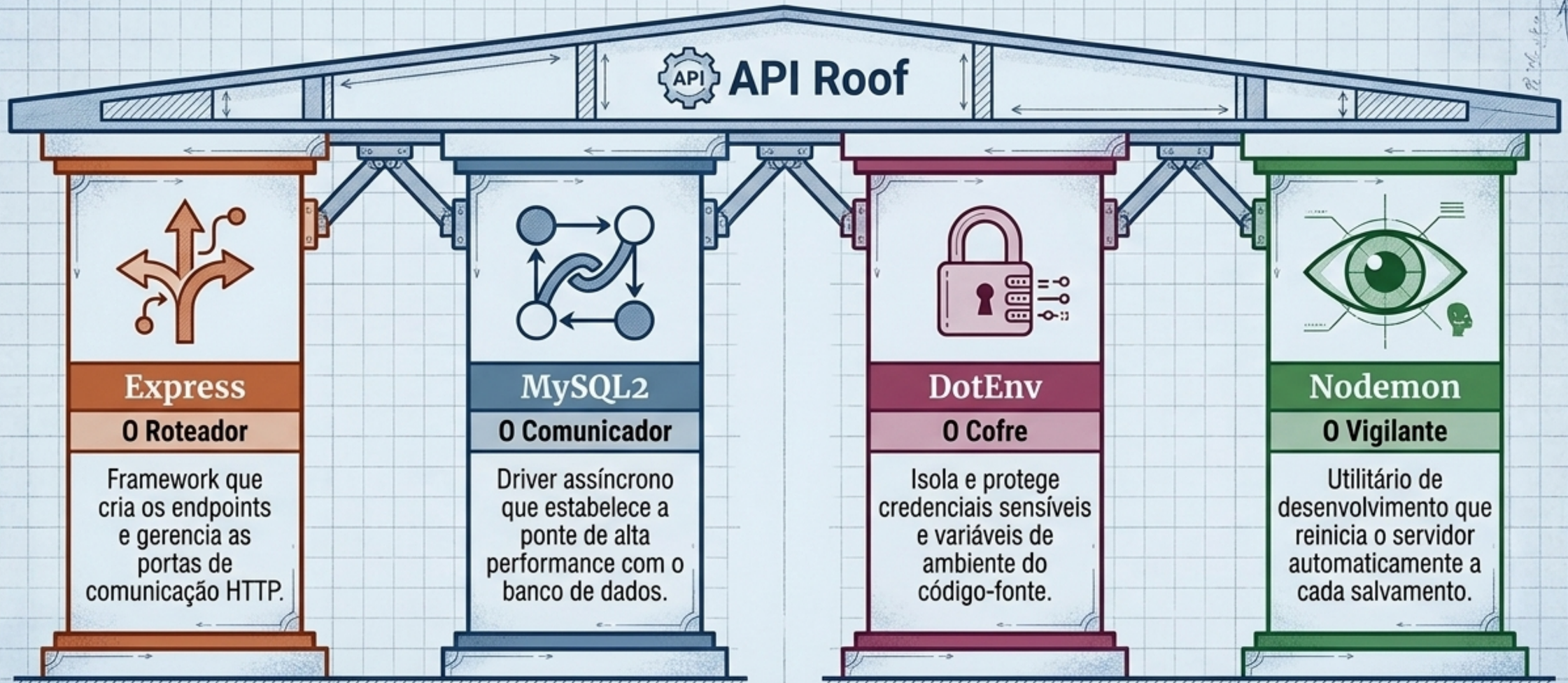
Uso de `DECIMAL(4,2)` no lugar de `FLOAT` para garantir precisão matemática exata (ex: 1.65, 45.00).

`CURRENT_TIMESTAMP`: O “print da hora”. Rastreabilidade automática gerada no exato momento da inserção.

Gemini Notebook

O Ecossistema de Dependências (NPM)

Os quatro pilares que sustentam a aplicação.



Gemini Notebook

Dissecando o package.json

O manual de instruções anatômico da sua aplicação.

```json
{
  "name": "api-nutri",
  "main": "index.js",
  "scripts": {
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "mysql2": "^3.0.0"
  }
}
```

A Porta Principal: O arquivo mestre que inicia o fluxo de execução da nossa aplicação.

O Atalho de Terminal: Criação do comando 'npm run dev' para automatizar o ambiente de desenvolvimento.

A Lista de Materiais: Permite que qualquer desenvolvedor instale todo o ecossistema com um único comando.

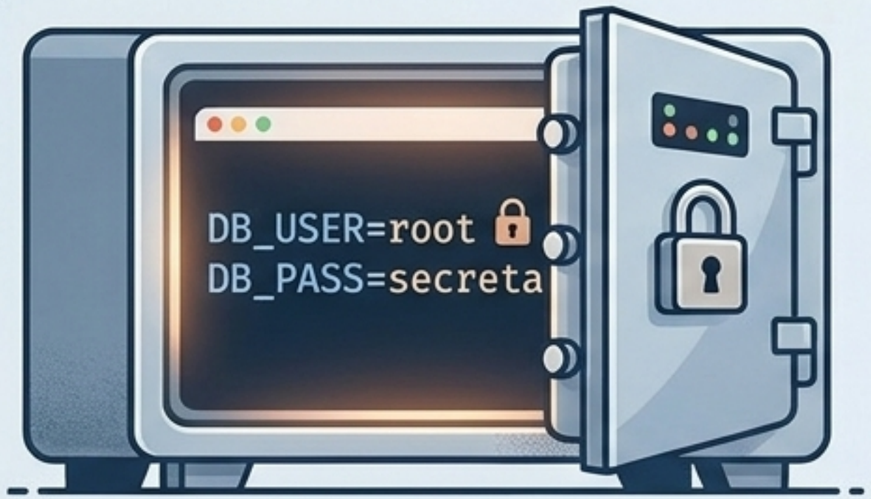
Gemini Notebook

Segurança Fase 1: Protegendo Segredos

Por que credenciais jamais devem ir para o GitHub.

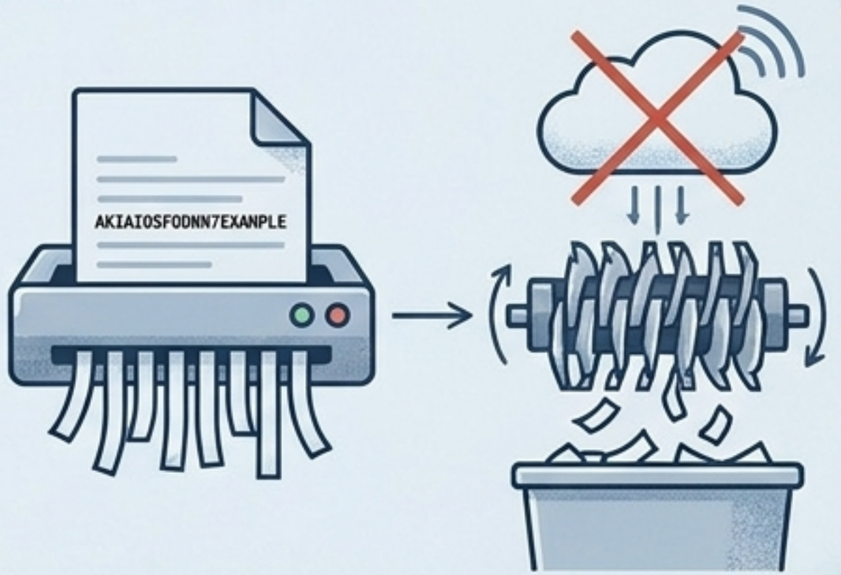
Robôs (scrapers) varrem plataformas públicas diariamente em busca de chaves de banco de dados *hardcoded*.

.env



O arquivo **.env** isola as credenciais localmente, garantindo que o código não contenha chaves fixas.

.gitignore

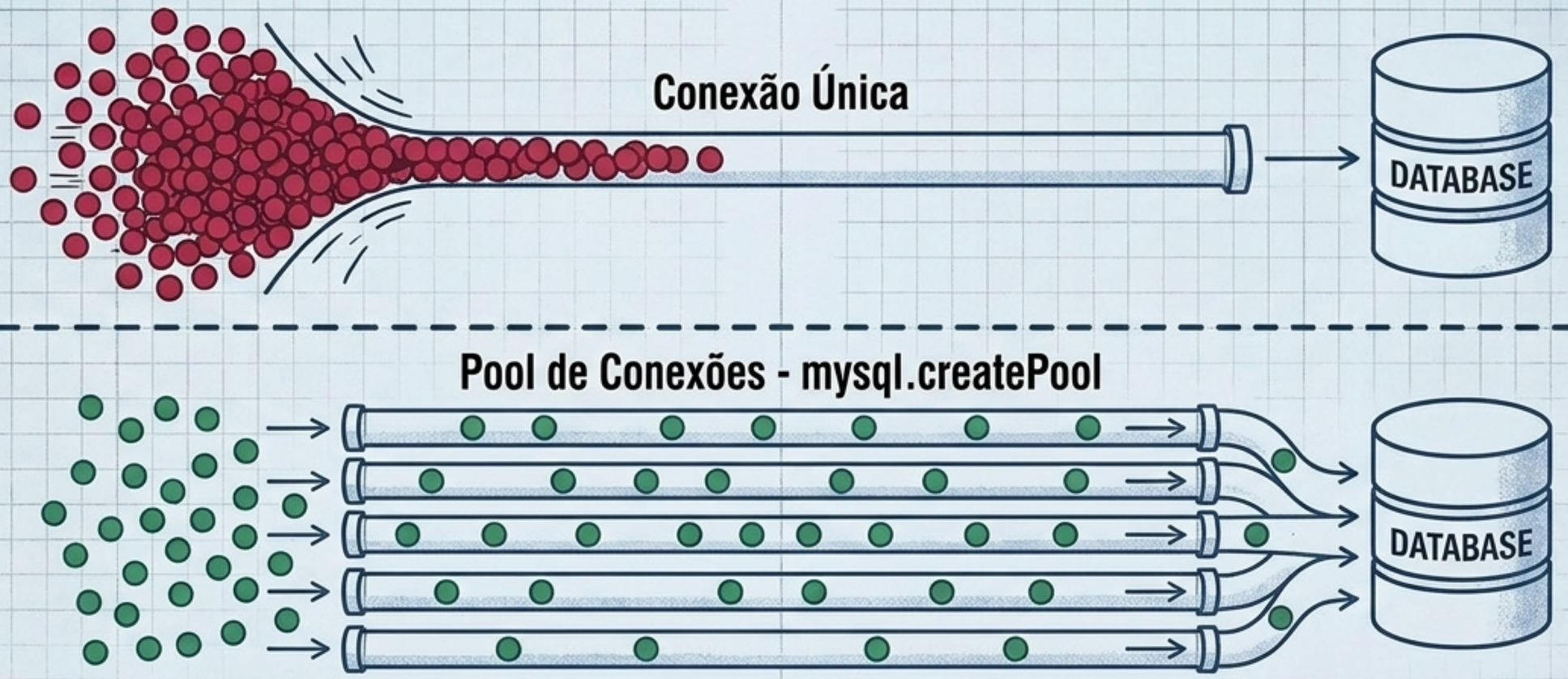


O **.gitignore** cega o sistema de versionamento, garantindo que os segredos nunca sejam enviados para a nuvem.

Gemini Notebook

O Motor de Conexão: Pool Dinâmico (`db.js`)

Evitando gargalos com a arquitetura do MySQL2.



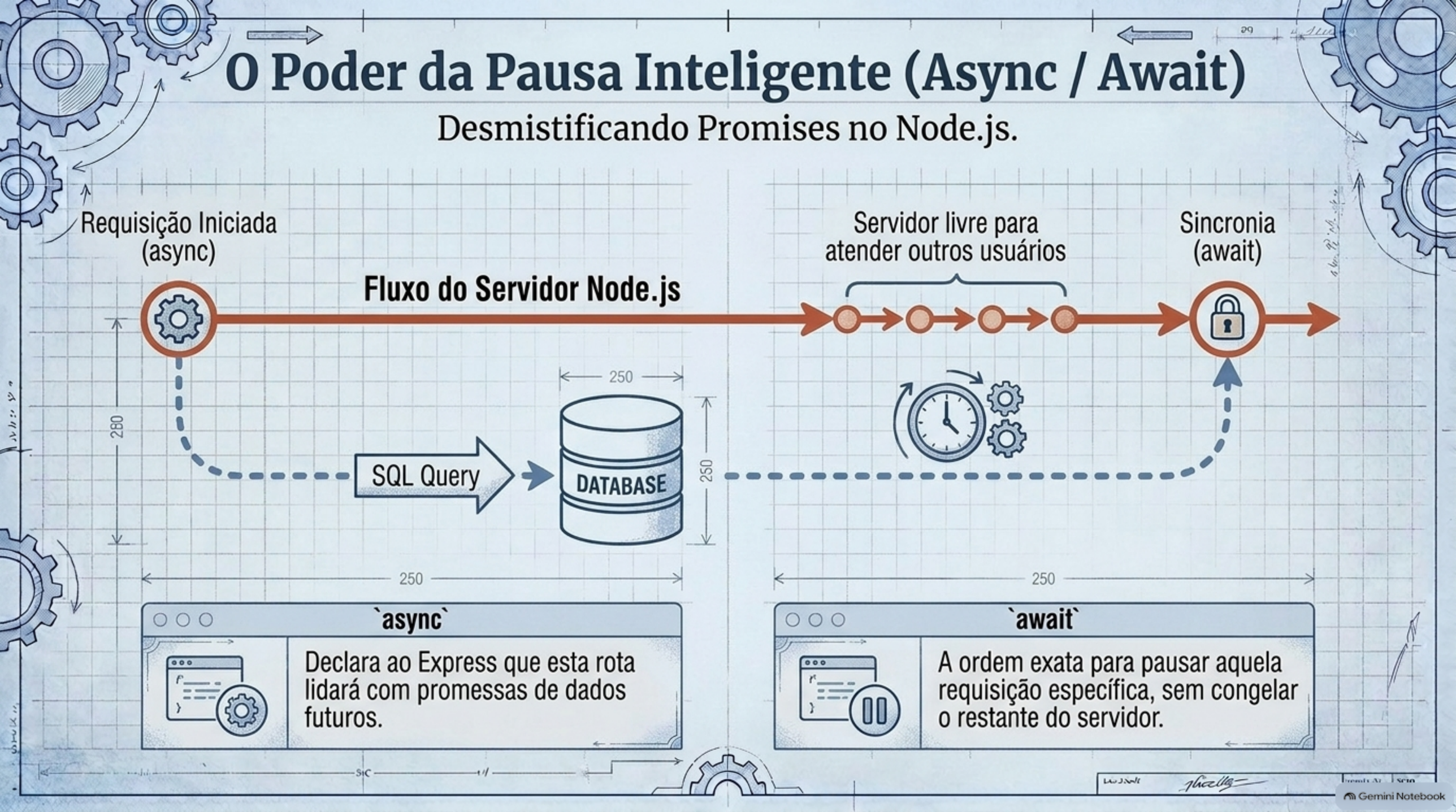
Processa uma requisição por vez. Sob demanda, causa sobrecarga, fila de espera e travamento do sistema.

Cria uma 'piscina' dinâmica. O sistema gerencia requisições simultâneas, abrindo e fechando pontes de forma inteligente.

Gemini Notebook

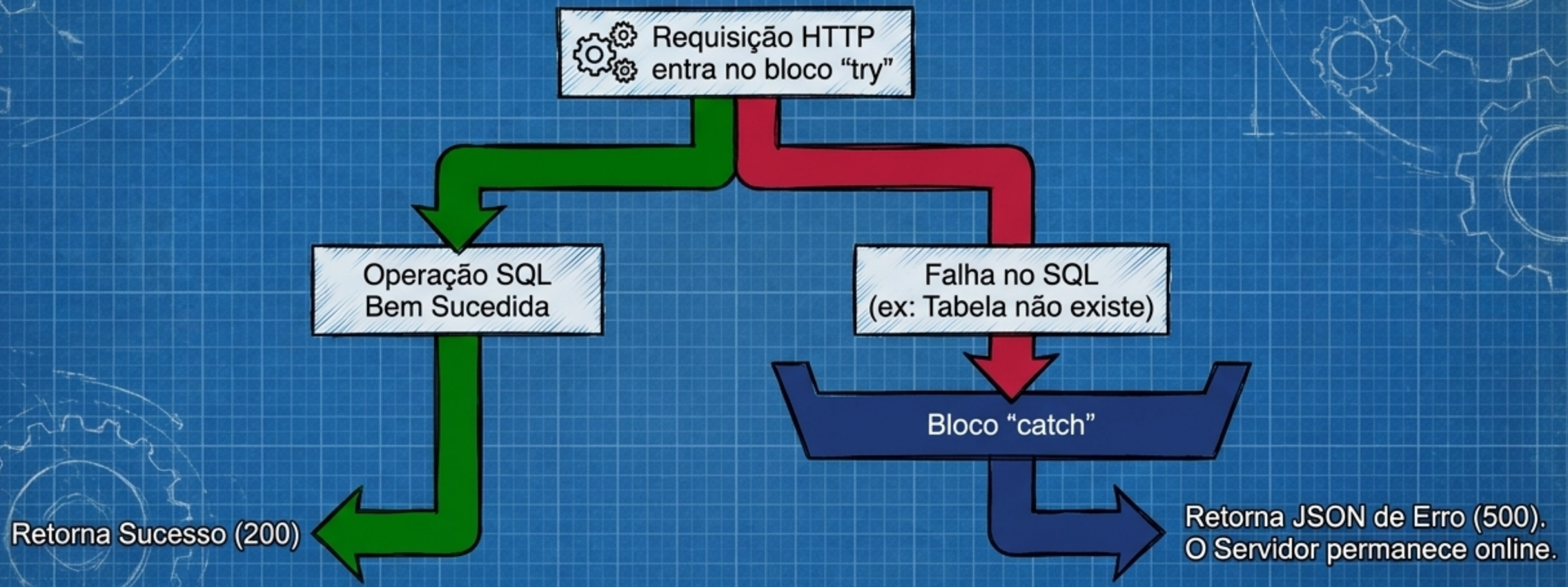
O Poder da Pausa Inteligente (Async / Await)

Desmistificando Promises no Node.js.



A Rede de Segurança: O bloco Try / Catch

Prevenindo a queda estrutural do back-end.



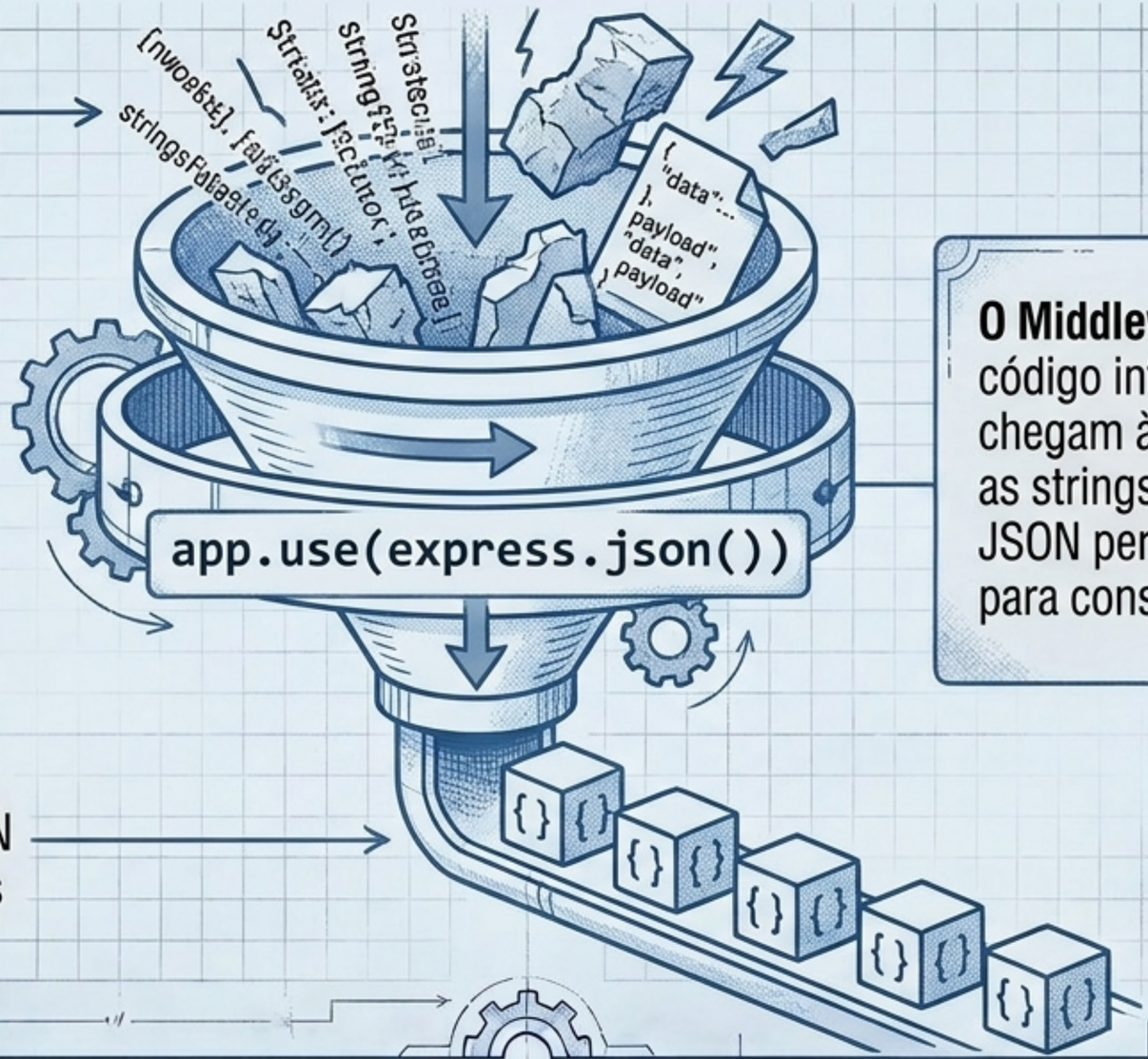
Sem o Try/Catch, uma falha de SQL derruba o servidor para todos os usuários. O Catch absorve o impacto e responde com dignidade.

Gemini Notebook

O Tradutor Universal (Express JSON Middleware)

Transformando dados brutos em pacotes padronizados.

Entrada:
Dados Brutos



Saída:
Objetos JSON
Estruturados

O Middleware: Esta única linha de código intercepta dados brutos que chegam à aplicação, processa as as strings e entrega um objeto JSON perfeitamente formatado para consumo imediato.

Gemini Notebook

Anatomia de um Endpoint de Leitura (GET)

Dissecando a rota principal da API.

2. O Diálogo HTTP: Request (o que o cliente pede) e Response (o que o servidor devolve).

```
app.get('/paciente', async (req, res) => {
  try {
    const [rows] = await db.execute('SELECT * FROM pacientes');
    res.status(200).json(rows);
  } catch (error) { ... }
});
```

1. O Endereço: Onde o cliente bate na porta.

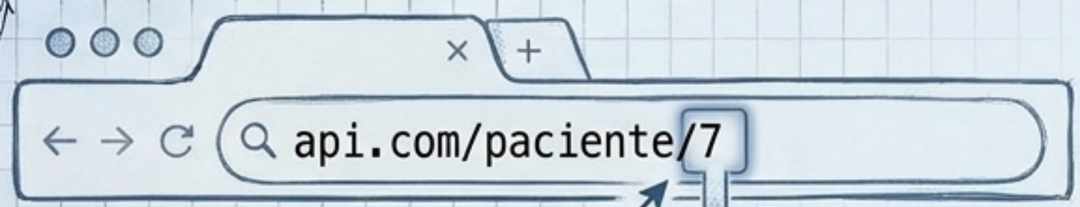
3. A Ação: Comando assíncrono que busca as linhas ('rows') no banco de dados.

4. A Entrega: A resposta empacotada no padrão universal JSON, acompanhada do carimbo de sucesso (200).

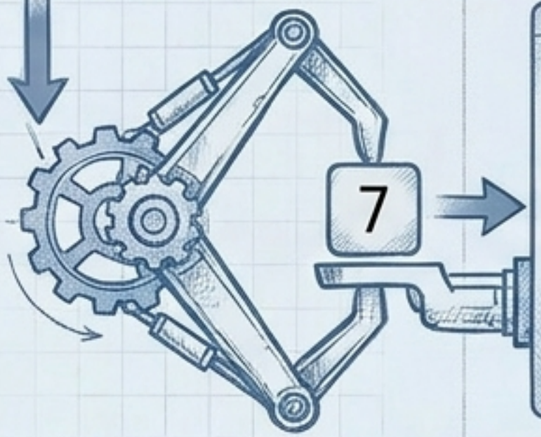
Gemini Notebook

Consultas Dinâmicas e Protegidas (Route Params)

Buscando dados específicos de forma segura.



Entrada (URL)



Node.js
Processador

`req.params.id`

7

```
SELECT * FROM pacientes
WHERE id = ?
```

Consulta SQL Segura

A Defesa contra SQL Injection: O uso da interrogação (?) força a separação estrita entre o código SQL e o dado digitado pelo usuário, protegendo o banco contra consultas maliciosas.

Gemini Notebook

Matriz de Códigos HTTP (O Semáforo da API)

Como o servidor sinaliza o resultado da operação.

Família 200: Sucesso

200 OK

O processamento ocorreu sem falhas. A rota foi acessada e os dados foram processados e entregues ao cliente corretamente.

Família 400: Erro do Cliente

404 Not Found

O usuário cometeu um erro de sintaxe, buscou um ID inexistente ou acessou uma rota que não existe. A responsabilidade é do front-end.

Família 500: Erro do Servidor

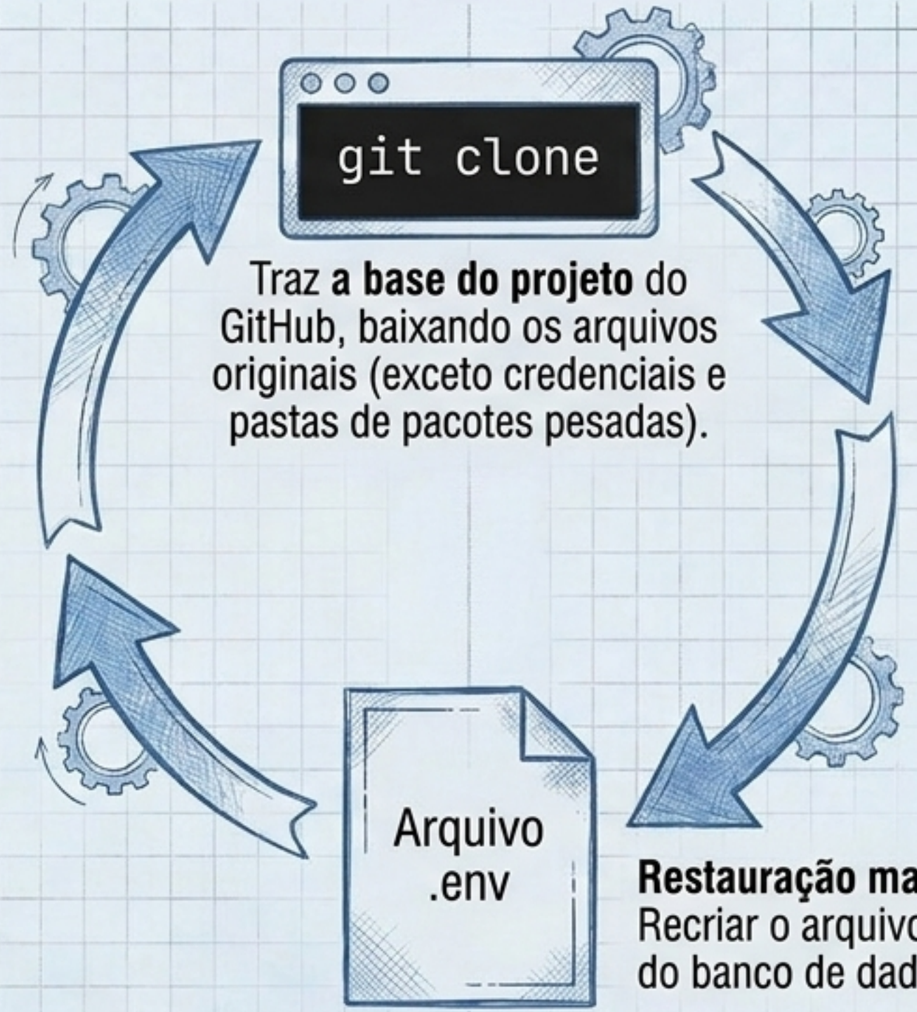
500 Internal Server Error

O banco de dados caiu, as credenciais estão incorretas ou há erro de sintaxe no código. O Try/Catch intercepta essa falha.

Gemini Notebook

Versionamento e o Ciclo de Clonagem

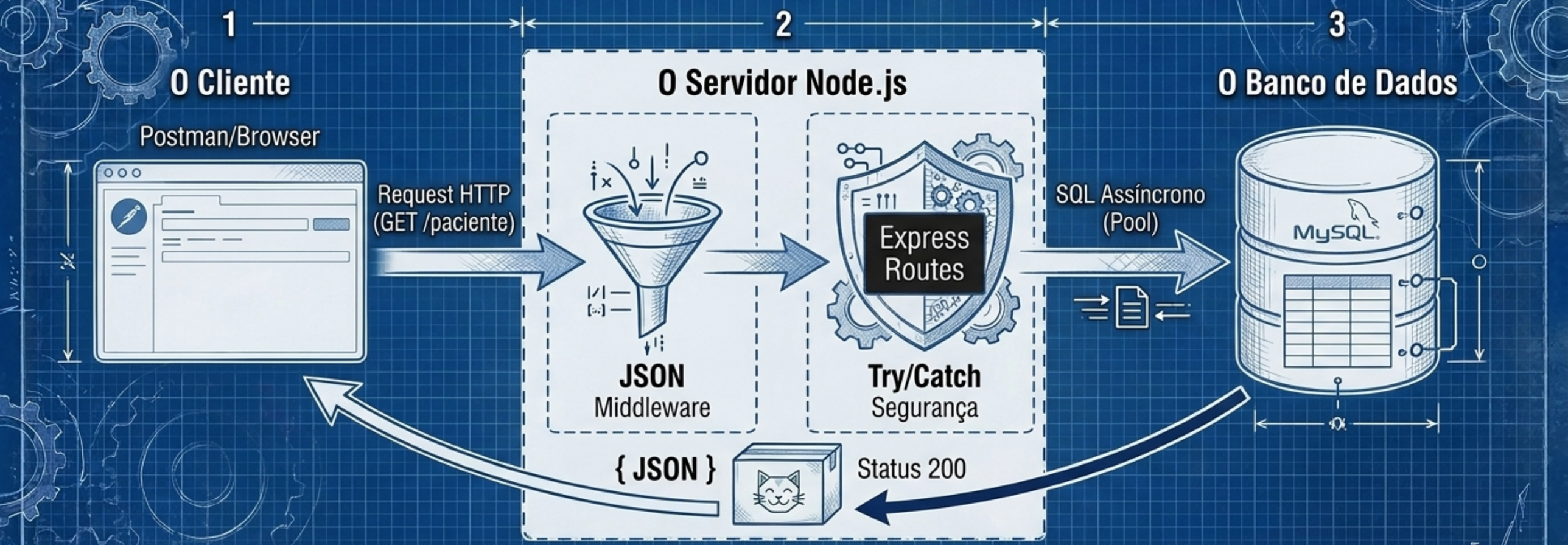
Como restaurar o projeto em uma nova máquina.



Gemini Notebook

A Arquitetura Finalizada

O Blueprint consolidado para produção.



A interligação perfeita entre roteamento dinâmico, sanitização de dados, pools assíncronos e tratamento defensivo cria uma aplicação de alta performance e à prova de falhas.

Gemini Notebook